

Algorithmique et Programmation orientée-objet

Ch.9 – Listes chaînées

Bruno Quoitin
(bruno.quoitin@umons.ac.be)

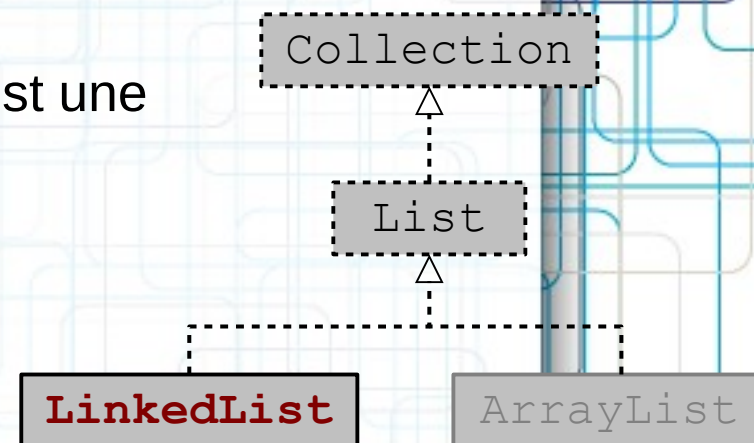
Table des Matières

- **Introduction**
- **Liste simplement chaînée**
 - Principes
 - Implémentation
 - Polymorphisme
- **Liste doublement chaînée**
- **Comparaison aux tableaux dynamiques**

Listes Chaînées

- **Introduction**

- Une **liste chaînée** est une structure de données qui met en oeuvre l'ADT Liste, i.e. une séquence d'éléments.
- Contrairement aux tableaux dynamiques, elle permet l'**insertion et la suppression efficaces d'éléments**, notamment en début/fin de liste.
- En revanche, elle est moins efficace que les tableaux dynamiques pour accéder à un élément situé à une position arbitraire.
- La classe `LinkedList` fournie par le JCF est une implémentation de liste chaînée.



Listes Chaînées

- Exemple

```
import java.util.LinkedList;
import java.util.List;

List<Carre> carres = new LinkedList<Carre>();
carres.add(new Carre(5, 7, -60, 5));
carres.add(new Carre(4, 1, 0, 2));
System.out.println("Taille = "+carres.size());

for (Carre c: carres)
    carre.deplacer(Math.random(), Math.random());

carres.set(0, new Carre(2, 2, 2, 2));
```

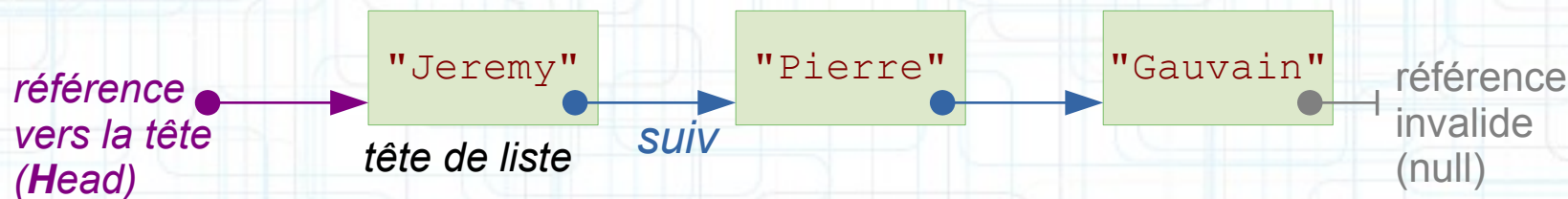
Table des Matières

- **Introduction**
- **Liste simplement chaînée**
 - Principes
 - Implémentation
 - Polymorphisme
- **Liste doublement chaînée**
- **Comparaison aux tableaux dynamiques**

Listes Chaînées

- **Principe**

- Une liste chaînée est une séquence de *nœuds*.
- Chaque nœud est un *objet* qui stocke
 - un *élément* ou la référence vers cet élément
 - un *lien* (référence) vers le nœud *suivant*



- Le premier nœud est appelé la *tête de la liste* (*head*). Conserver la *référence vers la tête* suffit à accéder à l'entièreté de la liste.
- Une telle liste est appelée *liste simplement chaînée*.

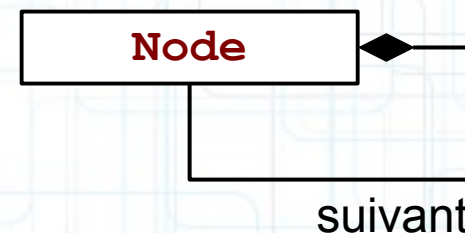
Listes Chaînées

- **Structure réursive**

- Une liste chaînée peut être définie par une *relation de récurrence*. Une liste l est
 - soit une liste vide $[]$
 - soit (n, r) où n est le nœud de tête suivi d'une liste r .
- Exemple : la liste 1, 2, 3 pourrait être écrite $(1, (2, (3, [])))$.

- **Structure auto-référentielle**

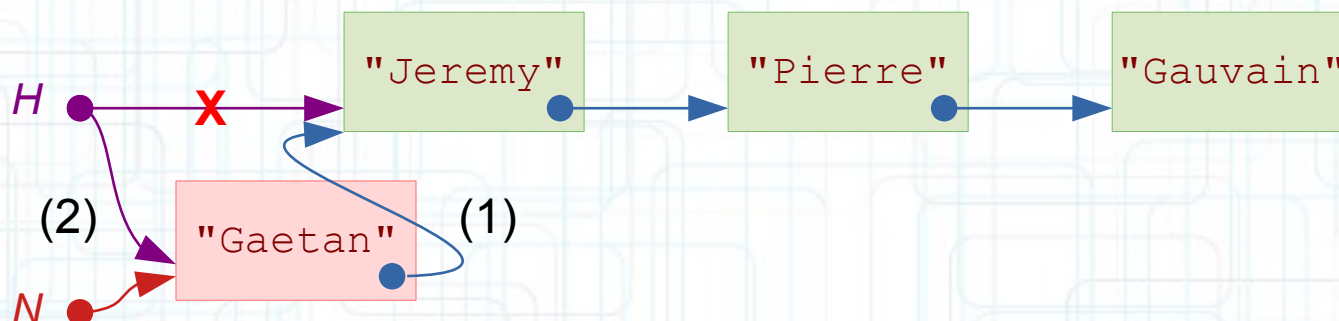
- Cela se reflète également dans les relations entre classes. Supposons que la classe `Node` soit utilisée pour représenter un nœud d'une liste chaînée. La classe `Node` contient une référence vers un objet de type `Node`.



Listes Chaînées

- **Principe**

- Ajout d'un élément en tête de liste

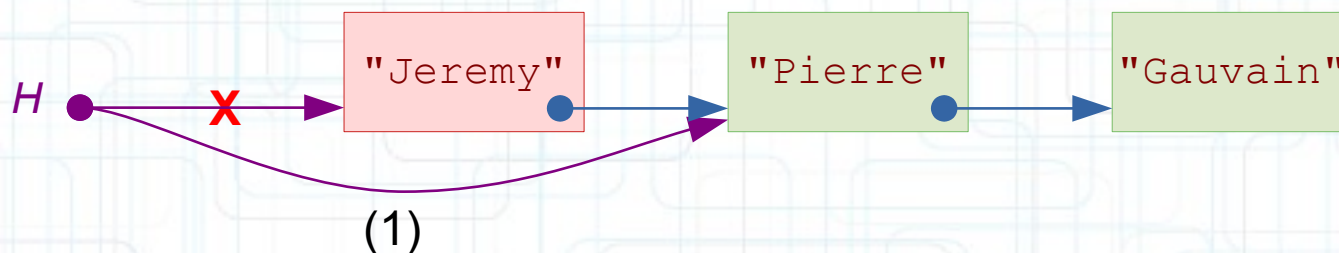


soit N nouveau nœud

(1) $N.\text{suiv} = H$

(2) $H = N$

- Suppression de l'élément en tête de liste



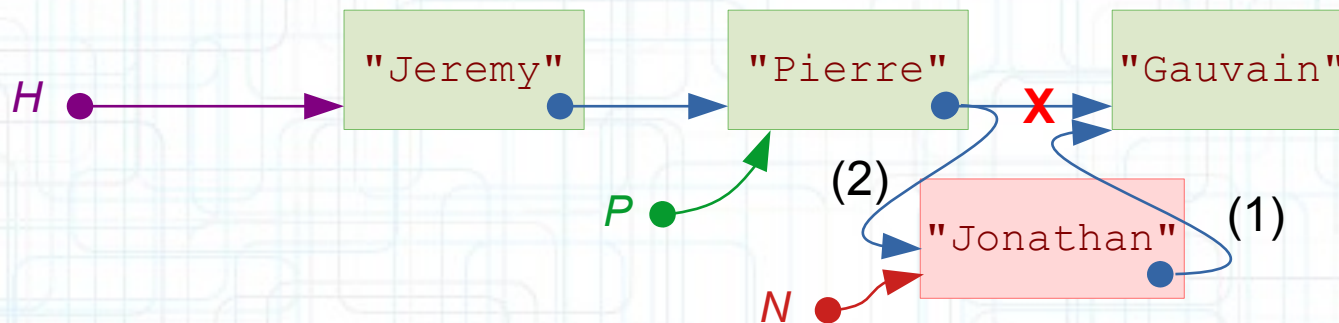
(1) $H = H.\text{suiv}$

- Ces deux opérations sont réalisées en temps constant.

Listes Chaînées

- **Principe**

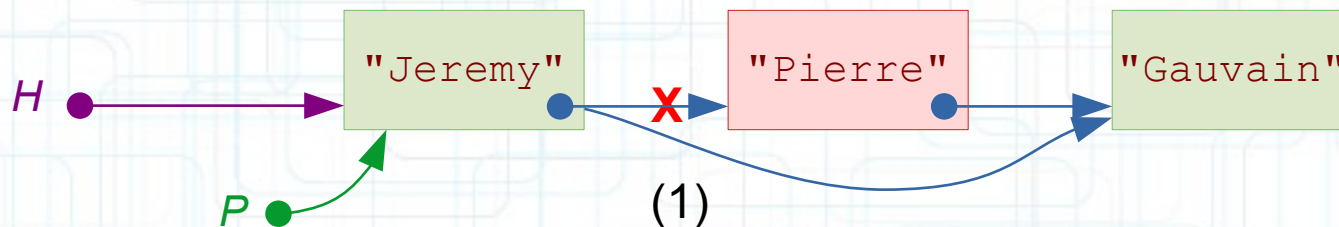
- Ajout d'un élément en « milieu » de liste



soit N nouveau nœud
 P prédécesseur

- (1) $N.\text{suiv} = P.\text{suiv}$
- (2) $P.\text{suiv} = N$

- Suppression d'un élément en « milieu » de liste



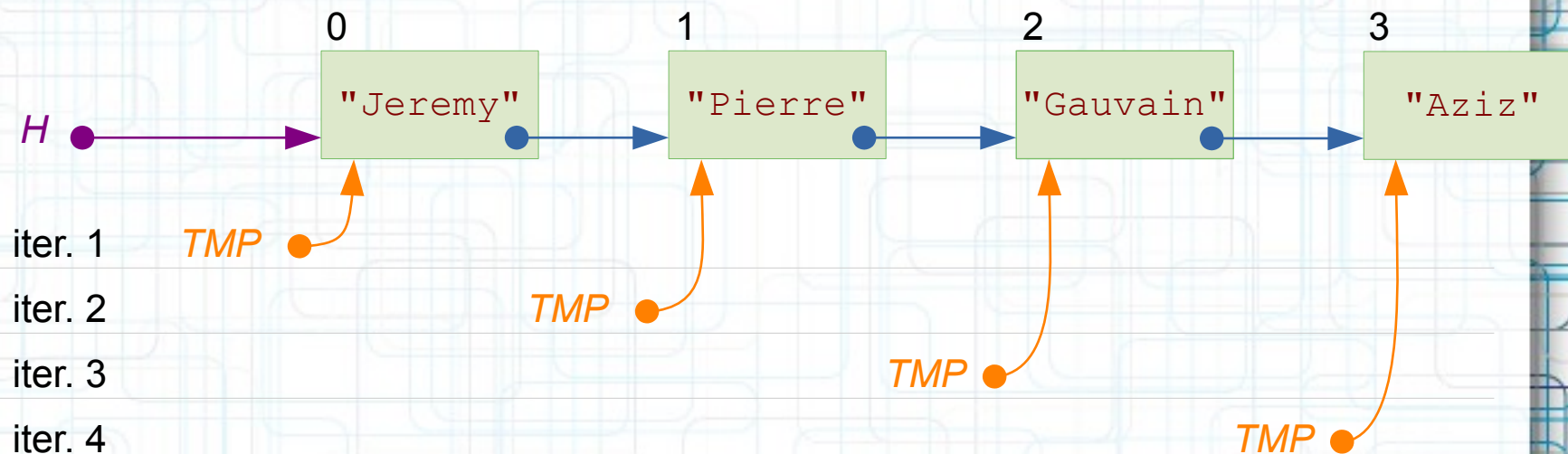
soit P prédécesseur

- (1) $P.\text{suiv} = P.\text{suiv}.\text{suiv}$

- Ces deux opérations sont réalisées en temps constant... à condition de connaître le prédécesseur P !

Listes Chaînées

- **Principe**
 - Parcourir tous les éléments



soit *TMP*, variable temporaire

TMP = *H*

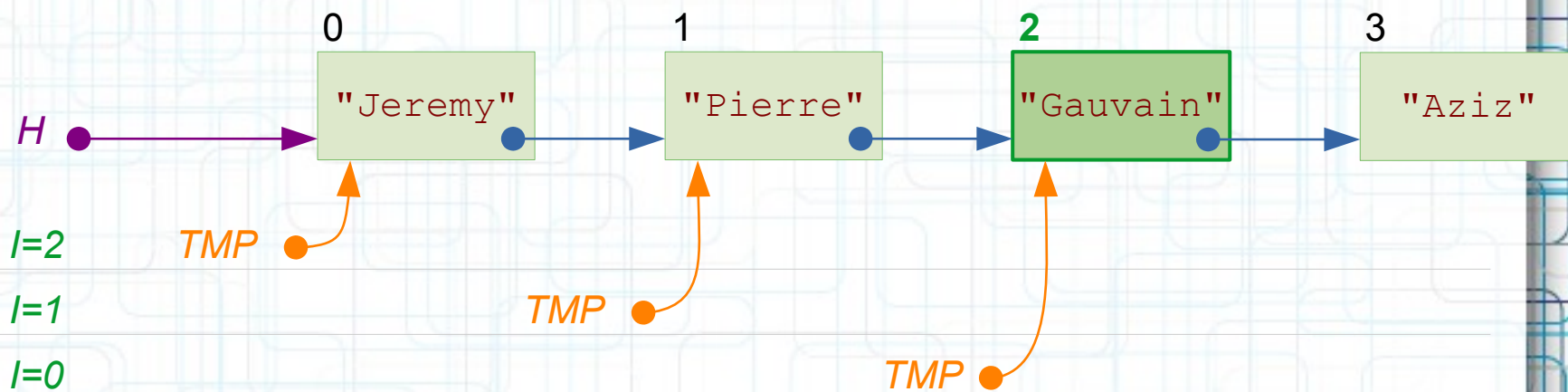
tant que (*TMP* != null) **faire**
consommer (*TMP*.element)

TMP = *TMP*.suiv

Listes Chaînées

- **Principe**

- Accès à un élément à une position arbitraire i avec $0 \leq i < N$



soit TMP , variable temporaire

$TMP = H$

tant que ($i > 0$) **faire**

$TMP = TMP.suiv$

$i = i - 1$

retourner $TMP.element$

Listes Chaînées

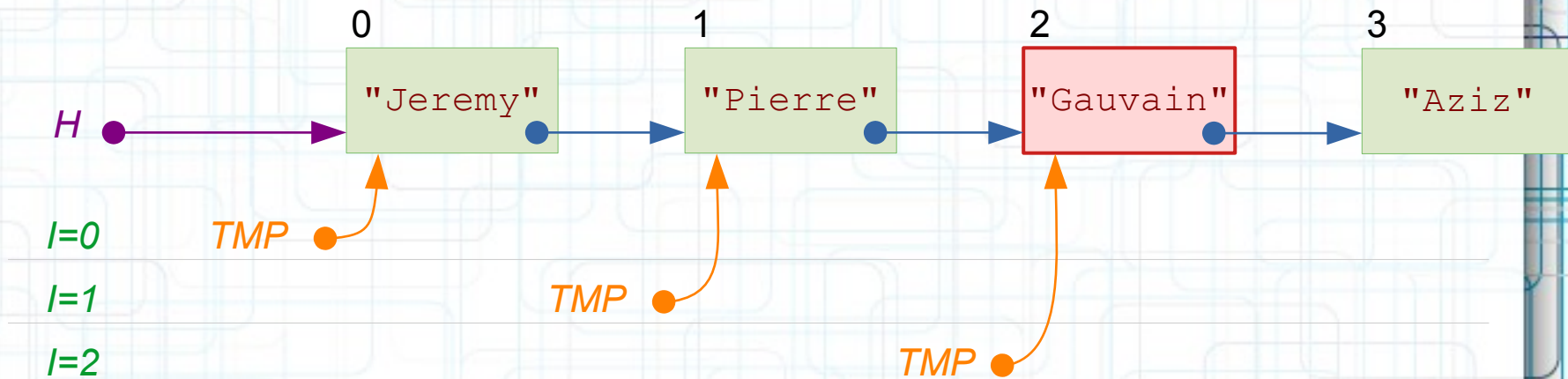
- **Principe**

- Rechercher la position d'un élément E
- Si l'élément n'est pas trouvé, la position retournée est -1

```

soit TMP, variable temporaire
    I, index

TMP = H
I = 0
tant que (TMP != null) faire
    si (E = TMP.element) alors
        retourner I
    TMP = TMP.suiv
    I = I + 1
retourner -1
    
```



Listes Chaînées

- **Principe**

- Ajouter/supprimer un élément en fin de liste ?
- Ajouter/supprimer un élément à une position arbitraire ?
- Déterminer la longueur de la liste ?
- Améliorations possibles
 - Maintenir la **longueur de la liste dans une variable** permet de diminuer le temps d'obtention de la longueur
 - Maintenir une **référence vers la fin** de la liste permet d'ajouter efficacement à la fin
 - Liste **doublement chaînée** : permet aussi de supprimer efficacement à la fin ; permet de diminuer le temps moyen d'accès par deux (en commençant par la tête ou la fin selon les cas)

Table des Matières

- **Introduction**
- **Liste simplement chaînée**
 - Principes
 - Implémentation
 - Polymorphisme
- **Liste doublement chaînée**
- **Comparaison aux tableaux dynamiques**

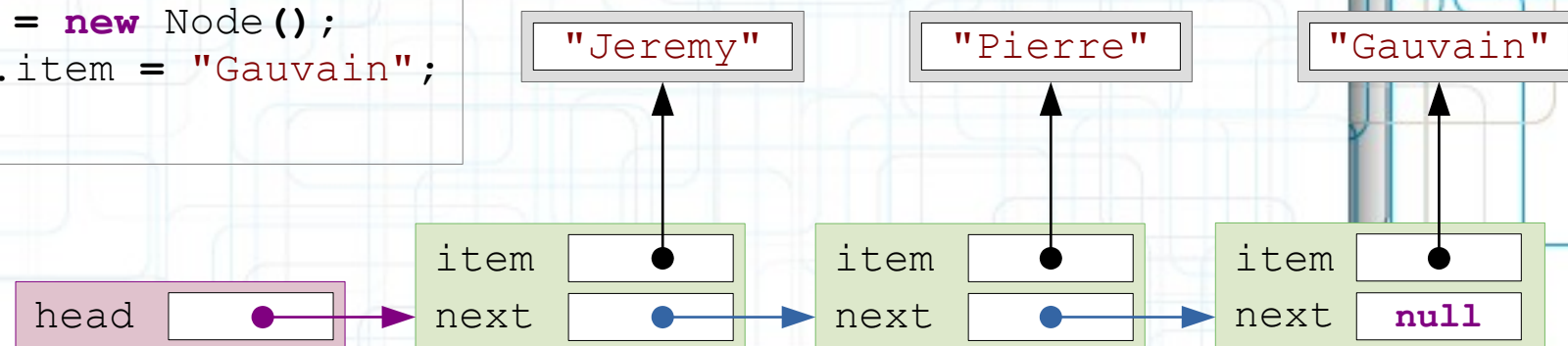
Listes Chaînées

- **Implémentation**

- L'élément de base d'une liste, un nœud, est représenté à l'aide de la classe `Node`, composée de deux variables
 - `item`, référence le **contenu**
 - `next`, est le **lien vers le nœud suivant**

```
public class Node {  
    public Object item;  
    public Node next;  
}
```

```
public static void main(...) {  
    Node head = new Node();  
    head.item = "Jeremy";  
    head.next = new Node();  
    head.next.item = "Pierre";  
    head.next.next = new Node();  
    head.next.next.item = "Gauvain";  
}
```



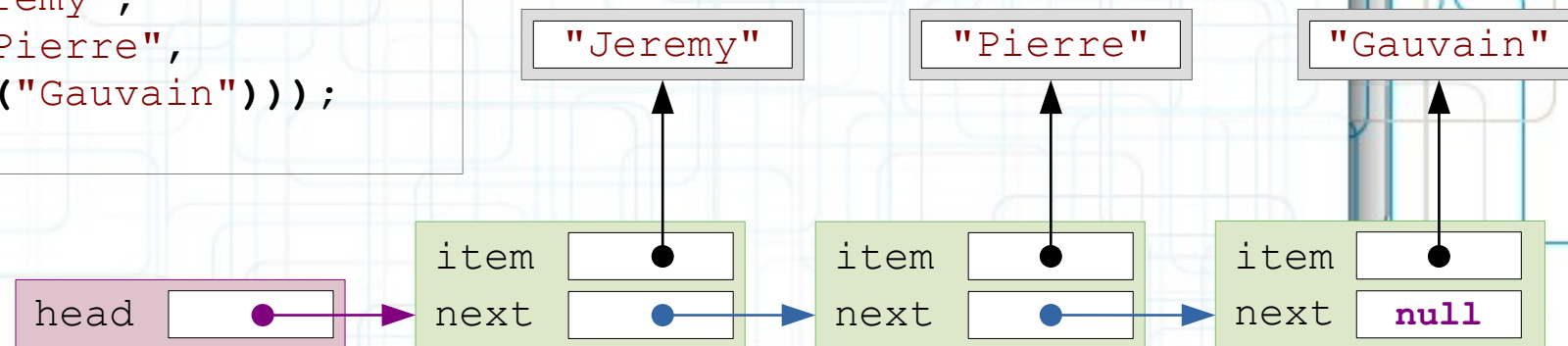
Listes Chaînées

- **Implémentation - variante**

- L'ajout de deux *constructeurs* à la classe Node permet de rendre la création de nœuds et de listes de nœuds très compacte.

```
public class Node {  
    public Object item;  
    public Node next;  
    public Node(Object item, Node next) {  
        this.item = item;  
        this.next = next;  
    }  
    public Node(Object item) {  
        this.item = item;  
    }  
}
```

```
public static void main(...) {  
    Node head =  
        new Node("Jeremy",  
            new Node("Pierre",  
                new Node("Gauvain")));  
}
```

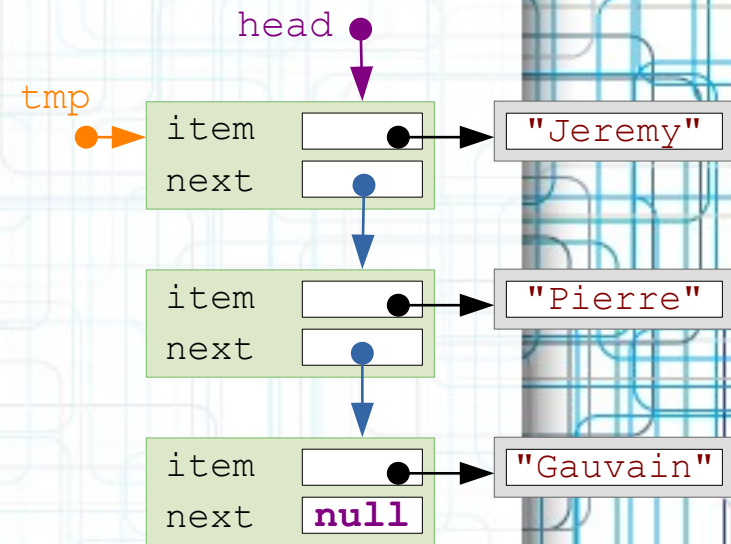


Listes Chaînées

- Implémentation

```
public static void main(...) {
    Node head = new Node();
    head.item = "Jeremy";
    head.next = new Node();
    head.next.item = "Pierre";
    head.next.next = new Node();
    head.next.next.item = "Gauvain";

    Node tmp = head;
    while (tmp != null) {
        System.out.println((String) tmp.item);
        tmp = tmp.next;
    }
}
```



➔ Attention, perdre la tête (de liste) est une erreur fréquente...

```
while (head != null)
    head = head.next;
```



Listes Chaînées

- **Encapsulation**

- La classe `LinkedList` permet de gérer l'ensemble de la liste
 - garder la référence vers la **tête**
 - maintenir la **taille** de la liste
 - rendre les services de l'ADT liste : ajout, suppression, accès aléatoire, etc.
 - cacher l'accès aux noeuds
- Cette approche permet d'assurer une **bonne encapsulation**.

```
public class LinkedList {  
    private class Node {  
        public Object item;  
        public Node next;  
    }  
  
    private Node head;  
    private int size;  
    (...)  
}
```

instances
de `Node` pas
accessibles
de l'extérieur

```
public static void main(...) {  
    LinkedList l = new LinkedList();  
    l.add("Jeremy");  
    l.add("Gauvain");  
    l.add(1, "Pierre");  
  
    for (int i = 0; i < l.size(); i++)  
        System.out.println(l.get(i));  
}
```

Listes Chaînées

- **Implémentation – Insertion en tête**

```
public class LinkedList {  
    (...)  
    public void add(Object item) {  
        Node n = new Node();  
        n.item = item;  
        n.next = head;  
        head = n;  
        size++;  
    }  
    (...)  
}
```

Toutes les méthodes de manipulation de liste qui suivent sont situées dans la classe `LinkedList`. Cependant, pour des raisons de place, cela ne sera pas indiqué explicitement.

Listes Chaînées

- Implémentation – accès aléatoire

recherche
du nœud i

```
public Object get(int i) {  
    if ((i < 0) || (i >= size))  
        throw new IndexOutOfBoundsException();  
  
    Node tmp = head;  
    while (i-- > 0)  
        tmp = tmp.next;  
  
    return tmp.item;  
}
```

```
public void set(int i, Object item) {  
    if ((i < 0) || (i >= size))  
        throw new IndexOutOfBoundsException();  
  
    Node tmp = head;  
    while (i-- > 0)  
        tmp = tmp.next;  
  
    tmp.item = item;  
}
```


Listes Chaînées

- Insertion à une position arbitraire

```
public void add(int i, Object item) {
    if ((i < 0) || (i > size))
        throw new indexOutOfBoundsException();
```

```
    if (i == 0) {
        Node n = new Node();
        n.item = item;
        n.next = head;
        head = n;
        size++;
        return;
    }
```

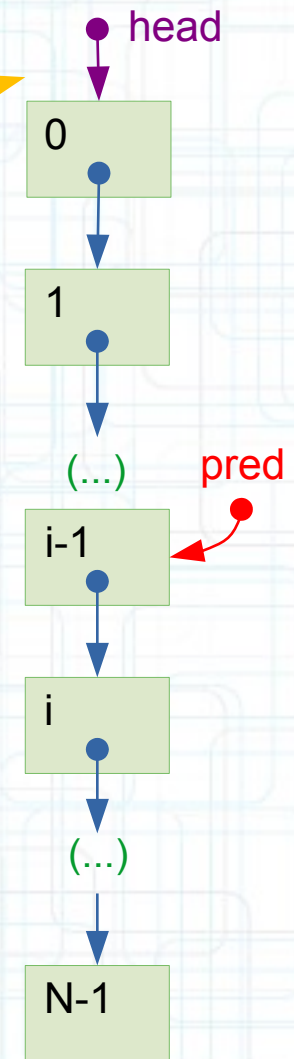
```
    Node pred = head;
    while (i-- > 1)
        pred = pred.next;
```

```
    Node n = new Node();
    n.item = item;
    n.next = pred.next;
    pred.next = n;
    size++;
}
```

recherche du
prédécesseur
du nœud i

insertion
en tête de liste
i = 0

insertion dans
la suite de la liste
 $i \in \{1, \dots, N\}$



Listes Chaînées

- Suppression à une position arbitraire

```
public void remove(int i) {
    if ((i < 0) || (i >= size))
        throw new indexOutOfBoundsException();

    if (i == 0) {
        head = head.next;
        size--;
        return;
    }

    Node pred = head;
    while (i-- > 1)
        pred = pred.next;

    pred.next = pred.next.next;
    size--;
}
```

recherche du
prédécesseur
du nœud i

suppression
de la tête de liste
i = 0

suppression
d'un élément
dans la suite
de la liste
 $i \in \{1, \dots, N-1\}$

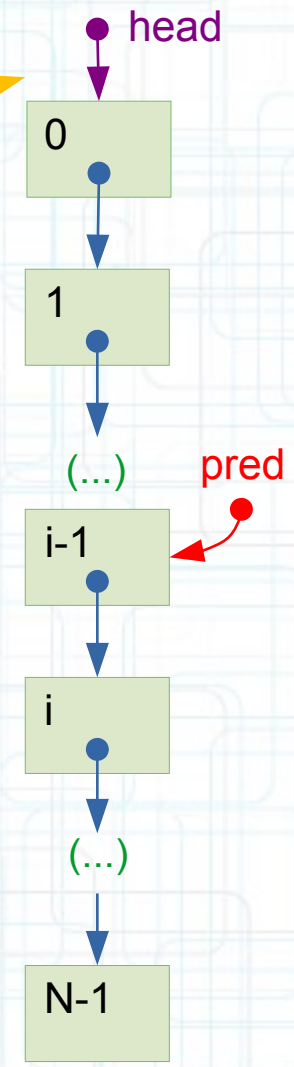


Table des Matières

- **Introduction**
- **Liste simplement chaînée**
 - Principes
 - Implémentation
 - Polymorphisme
- **Liste doublement chaînée**
- **Comparaison aux tableaux dynamiques**

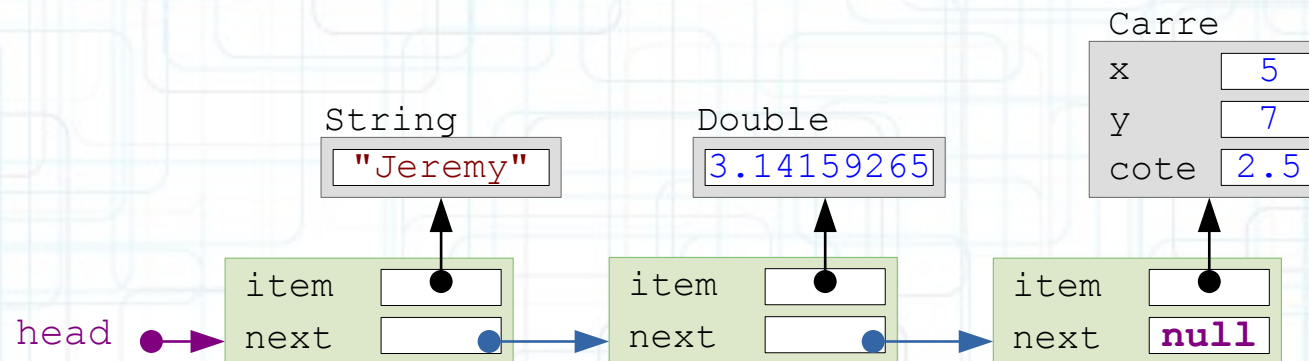
Listes Chaînées

- **Polymorphisme**

- Dans l'implémentation proposée, les *éléments* sont conservés dans les nœuds à l'aide de références de type `Object`.

```
public class Node {  
    public Object item;  
    public Node next;  
}
```

- Il s'agit de références *polymorphiques*, ce qui veut dire qu'elles peuvent référencer des instances de n'importe quelle classe en relation **is-a** avec `Object` ⇒ toutes les instances



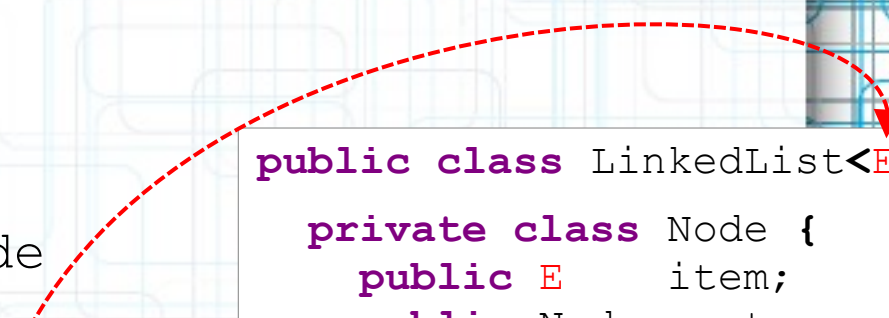
Listes Chaînées

- **Polymorphisme**

- Une autre approche se base sur le *polymorphisme paramétrique*. Dans l'exemple ci-contre, les éléments doivent être en relation *is-a* avec `E`, le paramètre de type.

```
public class Node<E> {  
    public E    item;  
    public Node next;  
}
```

- Attention, cependant lorsque `Node` devient une classe interne de `LinkedList`, le paramètre de type `E` doit seulement être défini sur la classe externe⁽¹⁾.



```
public class LinkedList<E> {  
    private class Node {  
        public E    item;  
        public Node next;  
    }  
  
    private Node head;  
    private int size;  
  
    public void add(E item);  
    public E get(int i);  
  
    (...)  
}
```

⁽¹⁾ la raison sera expliquée dans le chapitre portant sur les « generics », le polymorphisme paramétrique de Java..

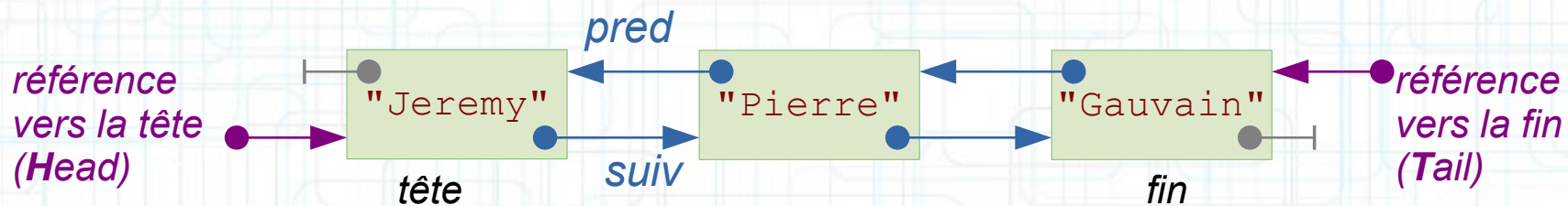
Table des Matières

- **Introduction**
- **Liste simplement chaînée**
 - Principes
 - Implémentation
 - Polymorphisme
- **Liste doublement chaînée**
- **Comparaison aux tableaux dynamiques**

Listes Chaînées

- **Liste doublement chaînée**

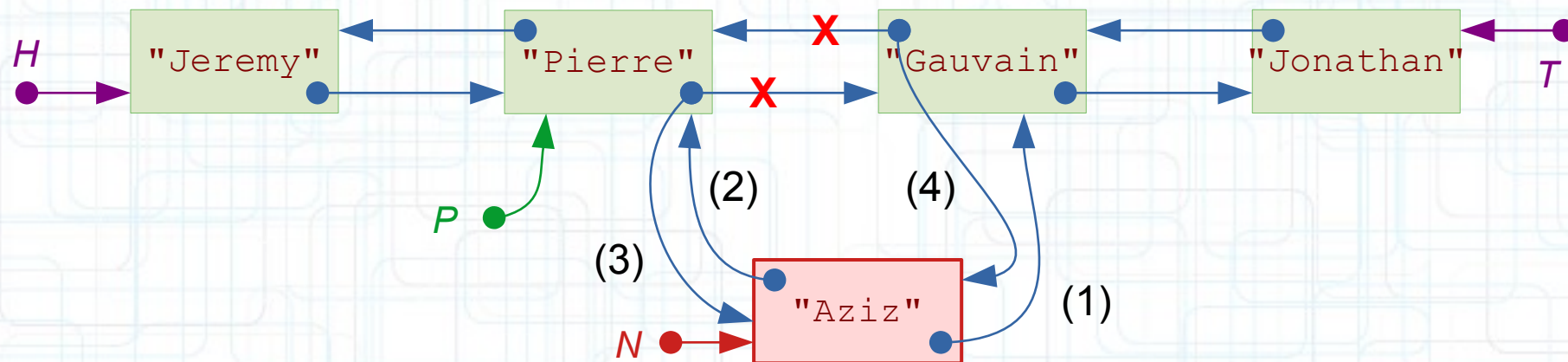
- Une **liste doublement chaînée** est composée de nœuds qui possèdent deux liens : l'un vers leur successeur (noté *suiv*), l'autre vers leur prédécesseur (noté *pred*).



- Les avantages d'une liste doublement chaînée sont
 - permet l'insertion et la suppression efficaces en fin de liste
 - permet de se déplacer d'avant en arrière dans la liste
 - divise par deux le temps d'accès dans le pire des cas (en commençant par la tête ou la fin, la moitié de la liste est à parcourir)

Listes Chaînées

- Liste doublement chaînée - Insertion



soit N nouveau nœud
 P prédécesseur

quid en début de liste ?

- (1) N .suiv = P .suiv
- (2) N .pred = P
- (3) P .suiv = N
- (4) N .suiv.pred = N

quid en fin de liste ?

Table des Matières

- **Introduction**
- **Liste simplement chaînée**
 - Principes
 - Implémentation
 - Polymorphisme
- **Liste doublement chaînée**
- **Comparaison aux tableaux dynamiques**

Listes Chaînées

- **ArrayList VS LinkedList**

- ArrayList et LinkedList sont deux implémentations de l'ADT Liste. Il est important de connaître leurs différences.

	ArrayList	LinkedList
ajout/suppression en fin	$O(1)$ en moyenne	$O(1)$
ajout/suppression en tête	$O(N)$	$O(1)$
insertion / suppression	$O(N)$	$O(N)$ $O(1)$ si préd./succ. connu ⁽¹⁾
taille	$O(1)$	$O(1)$
parcours séquentiel	$O(N)$	$O(N)$
accès aléatoire	$O(1)$	$O(N)$
surcoût mémoire par élément	0 (attention au taux d'occupation)	2 références (chacune de 32/64-bits)

⁽¹⁾ cas du parcours avec LinkedListIterator